Experiment No. 6

Title

AJAX-Based Search with Autocomplete Suggestions

Aim

To implement a real-time anime search system using AJAX and the JavaScript Fetch API that dynamically retrieves and displays multimedia data with autocomplete suggestions without reloading the webpage.

Software Required

• Text Editor (VS Code / Notepad)
• Web Browser (Chrome / Firefox)
• Internet Connection

Theory

AJAX (Asynchronous JavaScript and XML) is a web development technique used to send and receive data from a server asynchronously without reloading the webpage. It enables dynamic interaction between the client and the server by updating only specific parts of the webpage instead of refreshing the entire page.

Modern AJAX implementations commonly use the **Fetch API**, which provides a promise-based approach for making asynchronous HTTP requests.

In this experiment, a real-time search system is implemented using AJAX. When a user types in the search input field, the application sends an asynchronous request to a public anime API and retrieves data in **JSON format**. The results are then processed and displayed dynamically as autocomplete suggestions.

The system uses the **Jikan API**, which provides anime data sourced from the anime database platform MyAnimeList.

The experiment demonstrates advanced client-side techniques such as:

• Live search suggestions
• Predictive dropdown interface
• Multimedia API integration
• Debouncing to reduce API requests
• Dynamic DOM manipulation
• Event-driven programming

These techniques are widely used in modern web applications such as search engines, product recommendation systems, and streaming platforms.

Fetch API Syntax

```
fetch(API_URL)
.then(response => response.json())
.then(data => {
  // process data
});
```

OR using async/await:

```
const response = await fetch(API_URL);
const data = await response.json();
```

Explanation:

• **fetch()** – sends HTTP request to server
• **response.json()** – converts response into JSON format
• **await** – pauses execution until response is received

Debouncing Concept

Debouncing is a technique used to limit the number of API requests when a user types continuously.

Example Syntax:

```
clearTimeout(timer);

timer = setTimeout(()=>{
  searchFunction();
},300);
```

This ensures the search request is sent **only after the user pauses typing**, improving performance.

Procedure

1. Create an HTML file containing a search input field.
2. Design a suggestion dropdown container using CSS.
3. Write JavaScript code to capture user input events.
4. Implement the Fetch API to retrieve anime data from the Jikan API.
5. Use debouncing to prevent excessive API calls.
6. Dynamically generate suggestion cards using JavaScript.
7. Display anime posters, ratings, and episode counts.
8. Add click events to show detailed anime information.
9. Execute the program in a web browser.
10. Verify that search suggestions appear without page reload.

Program Code

index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Anime Live Search</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
<h2>Anime Search Engine</h2>
<input
type="text"
id="searchInput"
placeholder="Search Anime..."
autocomplete="off">
<div id="loader"></div>
<div id="suggestions" class="suggestions"></div>
</div>
<script src="script.js"></script>
</body>
</html>
```

style.css

```
body{
```

```
font-family:Arial;
background:#f4f6f9;
text-align:center;
padding:40px;
}
.container{
background:white;
width:600px;
margin:auto;
padding:25px;
border-radius:8px;
}
input{
width:90%;
padding:10px;
font-size:16px;
}
.suggestions{
margin-top:10px;
text-align:left;
}
.card{
display:flex;
align-items:center;
padding:10px;
border-bottom:1px solid #ddd;
cursor:pointer;
transition:0.2s;
}
.card:hover{
background:#eef2ff;
}
.card img{
width:50px;
height:70px;
margin-right:10px;
border-radius:4px;
}
.title{
font-weight:bold;
}
.rating{
font-size:12px;
color:#555;
```

```
}
#loader{
margin:10px;
font-size:13px;
color:#666;
}
```

script.js

```
const input = document.getElementById("searchInput");
const suggestions = document.getElementById("suggestions");
const loader = document.getElementById("loader");
let debounceTimer;
function debounceSearch(query){
clearTimeout(debounceTimer);
debounceTimer = setTimeout(()=>{
fetchAnime(query);
},400);
}
async function fetchAnime(query){
if(query === ""){
suggestions.innerHTML="";
return;
}
loader.innerText="Searching anime...";
try{
const response = await fetch(
`https://api.jikan.moe/v4/anime?q=${query}&limit=5`
);
const data = await response.json();
displayResults(data.data);
loader.innerText="";
}
catch(error){
loader.innerText="Error fetching data";
}}
function displayResults(animeList){
suggestions.innerHTML="";
animeList.forEach(anime => {
const card = document.createElement("div");
card.classList.add("card");
card.innerHTML=`
<img src="${anime.images.jpg.image_url}">
<div>
```

```
<div class="title">${anime.title}</div>
<div class="rating">
⭐ ${anime.score ?? "N/A"} | Episodes: ${anime.episodes ?? "?"}
</div>
</div>`;
card.addEventListener("click",()=>{
showDetails(anime);
});
suggestions.appendChild(card);
});
}
function showDetails(anime){
suggestions.innerHTML=`
<div style="padding:15px">
<h3>${anime.title}</h3>
<img src="${anime.images.jpg.large_image_url}" width="200">
<p><b>Episodes:</b> ${anime.episodes ?? "Unknown"}</p>
<p><b>Rating:</b> ${anime.score ?? "N/A"}</p>
<p><b>Synopsis:</b> ${anime.synopsis.slice(0,200)}...</p>
</div>`;
}
input.addEventListener("input",function(){
const query = this.value.trim();
debounceSearch(query);
});
```

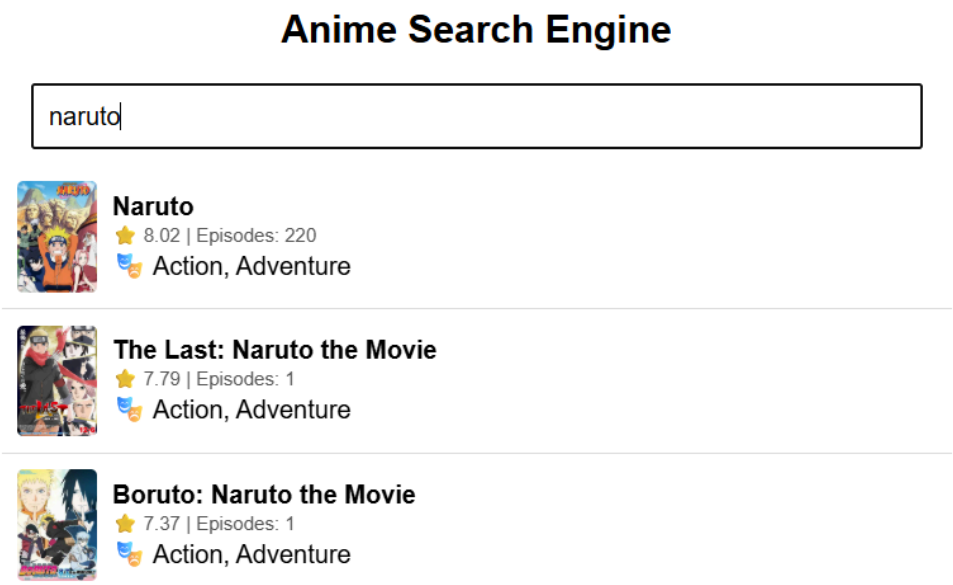
Student Tasks

Task 1

Modify the program to display **anime genre information** in the suggestion cards.

<u>script.js</u>

```
const genres = anime.genres
? anime.genres.slice(0,2).map(g => g.name).join(", ")
: "Unknown";
<div class="genres">
🎭 ${genres}
</div>
```

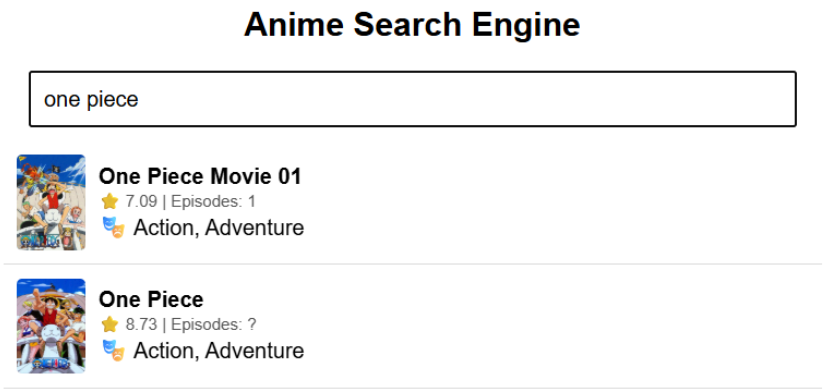


Task 2

Apply **CSS animations** to make suggestion cards appear smoothly.

Style.css

```
@keyframes fadeInUp {
  0% {
    opacity: 0;
    transform: translateY(10px);
  }
  100% {
    opacity: 1;
    transform: translateY(0);
  }
}
.card {
  display: flex;
  align-items: center;
  padding: 10px;
  border-bottom: 1px solid #ddd;
  cursor: pointer;
  transition: 0.2s;
  /* Animation properties */
  animation: fadeInUp 0.3s ease forwards;
}
```

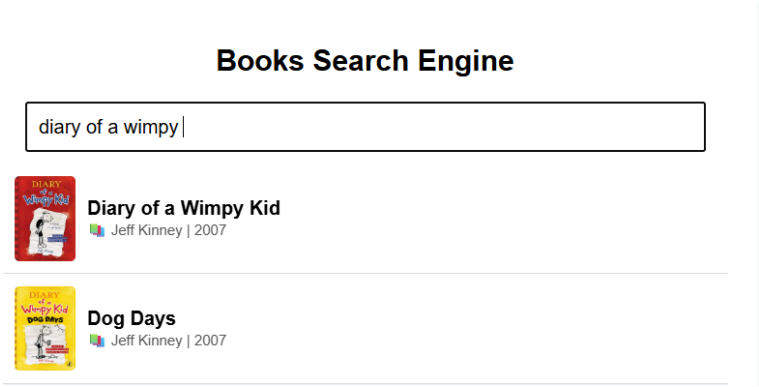


Task 3

Try this on a different API of your choice.

script.js

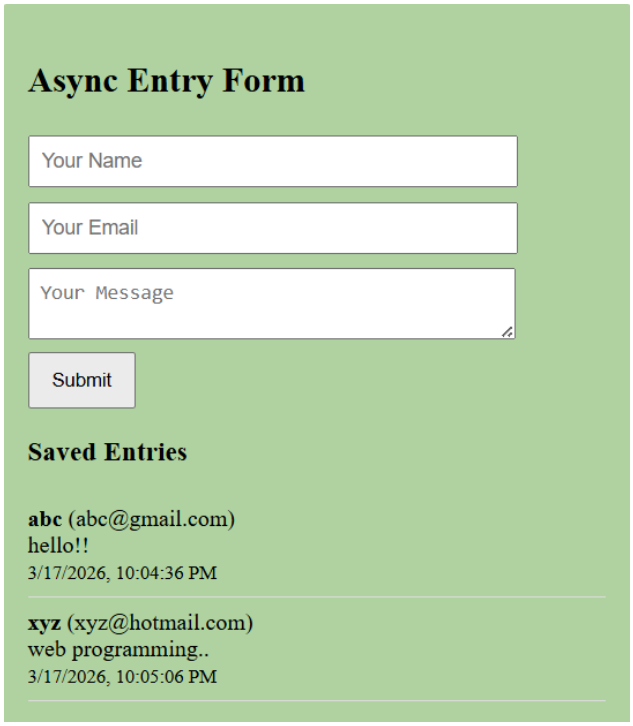
```
async function fetchBooks(query) {
  if (query === "") {
    suggestions.innerHTML = "";
    return;
  }
 loader.innerText = "Searching books...";
 try {
    const response = await fetch(
      `https://openlibrary.org/search.json?q=${encodeURIComponent(query)}&limit=5`
    );
    const data = await response.json();
    displayResults(data.docs);
    loader.innerText = "";
  } catch (error) {
    loader.innerText = "Error fetching data";
  }
}
```



Task 4

Create a form using save the entries asynchronously.

script.js

```
const form = document.getElementById("entryForm");
const entriesContainer = document.getElementById("entriesContainer");

let entries = JSON.parse(localStorage.getItem("entries")) || [];
displayEntries(entries);

form.addEventListener("submit", async function (e) {
  e.preventDefault(); // prevent page reload
  const name = document.getElementById("name").value.trim();
  const email = document.getElementById("email").value.trim();
  const message = document.getElementById("message").value.trim();
  if (!name || !email || !message) return;
  const entry = { name, email, message, timestamp: new Date().toISOString() };

  await saveEntryAsync(entry);

  form.reset();

  entries.push(entry);
  displayEntries(entries);
});

function saveEntryAsync(entry) {
  return new Promise((resolve) => {
    setTimeout(() => {

      localStorage.setItem("entries", JSON.stringify([...entries, entry]));
      resolve();
    }, 200); // simulate network delay
  });
}

function displayEntries(entriesList) {
  entriesContainer.innerHTML = "";
  entriesList.forEach((entry) => {
    const div = document.createElement("div");
    div.classList.add("entry");
    div.innerHTML = `
      <b>${entry.name}</b> (${entry.email}) <br>
      ${entry.message} <br>
```

```
    <small>${new Date(entry.timestamp).toLocaleString()}</small>
  `;
  entriesContainer.appendChild(div);
 });
}
```